

Module 19: Capstone Project 2

Introduction to C++ Programming — Part 2: Intermediate C++

Module Overview

The second and final capstone, staged into three commits exactly like Part 1's, now combining all nine Part 2 concepts (pointers through templates) into one real, track-based program. As with Part 1's capstone, your role shifts entirely to supervising, unblocking, and code-reviewing - there is no new material to lecture here. Treat this as a multi-session studio/lab block.

Suggested Duration: 2–3 sessions of 60–90 minutes each, or equivalent lab/studio time

Learning Objectives

By the end of this module, students will be able to:

- Design and implement a polymorphic class hierarchy from a written specification.
- Apply every major Part 2 concept (pointers, inheritance, polymorphism, operator overloading, file I/O, exceptions, STL containers/algorithms, templates) in one cohesive program.
- Persist records to a file that genuinely accumulates across separate program runs.
- Reject invalid/unsafe input with exceptions instead of crashing.
- Present and justify a completed program's design choices, as in the Part 1 capstone.

Prerequisites

- All of Modules 10-18. This module assumes fluency, not just familiarity, across the whole of Part 2.

Materials & Instructor Prep

- 19-capstone-project-2/README.md and the three track starter files, available to every student.
- A rubric tied explicitly to the three stages and the concept map in the README, so partial completion is easy to assess fairly.

Key Vocabulary

Term	Definition
Concept map (course term)	The table in the capstone README tracing each requirement back to the specific module that taught it - use it as your own facilitation anchor, not just a student reference.

Lesson Outline & Instructor Talking Points

Session 1 — Stage 1: The class hierarchy (Modules 10-12)

- Open with the concept map, exactly as in Part 1's capstone - reframe this project as assembly of known parts, not new material. This reassurance matters even more here, given how dense Modules 10-12 felt on first pass.
- Have students build their track's base class and 2-3 derived classes with virtual methods and a virtual destructor, storing them in a `vector<Base*>` built with `new`, printed with one polymorphic loop.
- Require the Stage 1 commit before ending the session.

Instructor note: Watch specifically for a missing virtual destructor and for accidental pass-by-value of polymorphic objects (object slicing) - both are exactly the Module 11/12 gotchas resurfacing under new project pressure.

Session 2 — Stage 2: File I/O, exceptions, STL container (Modules 13-16)

- Have students add a CSV logger with `std::ios::app` (Module 14), throw/catch exceptions for invalid values and unsafe thresholds (Module 15), and rebuild any manual search logic around a `std::map` keyed by name/ID (Module 16).
- If a track's Stage 2 also calls for a value-like class (e.g. Dosage), this is the moment to wire in its Module 13 operator overloads if the student built them.
- Require the Stage 2 commit.

Instructor note: This is the densest single stage - if a session is running short, it's reasonable to split Stage 2 itself across two sessions (file I/O + exceptions, then the STL container) rather than compress it.

Session 3 — Stage 3: Algorithms and templates (Modules 17-18)

- Have students apply STL algorithms (`max_element/min_element`, `count_if`, `transform`) to their stored history, and feed it into a `Statistics<T>` template - reused directly from Module 18's project if they already built one.
- Close with the same short (2-3 minute) informal presentations as Part 1's capstone: each student demos their program and explains one design decision.

- Require the final commit.

Instructor note: *The presentation step is worth protecting even under time pressure - it's the only place in the whole course where students practice articulating a design choice out loud, not just implementing one.*

Common Student Misconceptions

Misconception	How to Address It
"This capstone is unrelated to the Part 1 capstone - a fresh start."	For the Generic track especially, reframe it explicitly as an EXTENSION of the Part 1 Student Records system, not a new project - the continuity is deliberate course design, not a coincidence to downplay.
"I should build everything in one sitting and commit once at the end, like before."	Exactly as in Part 1's capstone, require the three separate staged commits - the habit-reinforcement is as much the point as the code itself.
"The three tracks are graded differently / one is 'easier' than the others."	All three exercise an identical set of nine concepts from the concept map - grade against that shared map, not against a perception of relative difficulty.

Exercises & Module Project

The capstone itself IS the assignment - no separate exercises. See 19-capstone-project-2/README.md for the full three-stage, three-track specification and the matching starter files.

Facilitation guidance: Grade holistically against the nine-row concept map, not a single fixed 'correct' output - value clean polymorphic design, real exception safety, and genuine cross-run file persistence as much as raw feature completeness.

Discussion & Comprehension Check Questions

1. Which of the nine Part 2 concepts (concept map) felt hardest to apply under this project's pressure, and why?

Guidance: No single right answer - use this as a genuine temperature check; pointers/polymorphism and the STL-container/algorithm combination are the two areas most students name.

2. How did you decide what belonged in your base class versus each derived class?

Guidance: Shared data/behavior every derived type needs belongs in the base; anything that differs meaningfully per type (a formula, a unique field) belongs in the derived class - there's genuine judgment here, not one fixed answer.

Assessment Ideas

- Use the three staged commits as a lightweight process rubric, exactly as in Part 1's capstone, alongside a functional correctness check against the concept map.
- The closing presentation doubles as an informal oral assessment of whether a student can justify their class hierarchy's design, not just get it to compile.

Answer Key Notes

There is no single solutions file for this capstone by design - it's an open-ended synthesis project, exactly like Part 1's. Use the concept map and three-stage checklist in 19-capstone-project-2/README.md as your grading anchor.

Wrap-Up & Bridge to Next Module

Close BOTH parts of the course here, not just this module. Point back at the very first roadmap from Module 0 and name what's now true: every student has TWO real, staged, committed projects on their own GitHub profile, spanning foundational and intermediate C++. Name the capstone README's explicit next steps (smart pointers, multiple inheritance, move semantics, multi-file projects with CMake, unit testing) as an honest, motivating map of what's next - not a vague send-off.

Instructor Tips

- Protect studio/lab time above all else across all three sessions of this module - your value here is walking the room, unblocking stuck students, and asking design questions, not lecturing, exactly as in Part 1's capstone.
- If your group included a strong mix of tracks, consider a short cross-track show-and-tell in the final session - seeing a Complex impedance class next to a Dosage class next to a Statistics<T> reused three different ways is a genuinely effective way to make the shared underlying concepts visible.